
A Comparative Analysis of Transformer and LSTM Architectures in Part-of-Speech Tagging

by

Anton Gamaliy — 2800529

Sami Finkbeiner — 2797796

Dan Gavriluta — 2808733

Abstract

This study compares Transformer and Bidirectional Long Short-Term Memory (BiLSTM) architectures for Part-of-Speech (POS) tagging. While Transformers are widely used in large-scale language modeling, their effectiveness on structured sequence labeling tasks remains less clear. We evaluate both models across different dataset sizes and hyperparameter settings using the Universal Dependencies (UD) English EWT corpus, with additional training on the Brown corpus. Our results show that BiLSTMs achieve higher overall performance, particularly in smaller data settings, while Transformers demonstrate more stable training and stronger performance gains as dataset size increases. These findings highlight a trade-off between data efficiency and scalability, suggesting that model selection for POS tagging should depend on the size of the available training data.

1 Introduction

Human language is inherently complex, with meaning often determined by both local context and long-range dependencies. One way this complexity manifests in Natural Language Processing is through Part-of-Speech (POS) tagging, where each word must be assigned a grammatical category such as noun, verb, or adjective [10]. Accurate POS tagging requires models to capture both local patterns, such as immediate word neighbors, and long-range dependencies, where the correct tag depends on distant words in a sentence [2]. Traditional Recurrent Neural Networks (RNNs) often struggle with long-range dependencies due to the vanishing gradient problem. Long Short-Term Memory networks (LSTMs), particularly bidirectional variants (BiLSTMs), mitigate this issue with gated mechanisms that preserve information across sequences, yet they can still underperform on rare or imbalanced POS categories [4]. In contrast, Transformer architectures employ multi-head self-attention to allow every token to interact directly with all others, providing a mechanism to model long-range syntactic relationships without sequential recurrence [11]. This paper explores the comparative performance of BiLSTM and Transformer architectures on Part-of-Speech tagging, evaluating their ability to capture long-range dependencies and handle class imbalance in natural language datasets. This study contributes by implementing a custom Transformer encoder, evaluating scaling behavior across dataset sizes, and analyzing attention-based interpretability for POS tagging.

2 Background

2.1 Part-of-Speech (POS)

Part-of-Speech (POS) tagging is a foundational task in Natural Language Processing (NLP) that involves assigning a grammatical category—such as noun, verb, or adjective—to each word in a sequence. Because the syntactic role of a word often depends on its neighbors, POS tagging is treated

as a sequence labeling problem. Accurate tagging is crucial for downstream applications, including dependency parsing, sentiment analysis, and machine translation, as it provides the structural backbone necessary for a machine to understand human syntax [10].

2.2 Recurrent Models: BiLSTM

Bidirectional Long Short-Term Memory (BiLSTM) networks are a widely used architecture for sequence labeling tasks such as POS tagging. By processing the input sequence in both forward and backward directions, BiLSTMs capture contextual information from both past and future tokens. This bidirectional structure makes them particularly effective at modeling local dependencies and short-range syntactic patterns. However, their sequential nature can limit their ability to efficiently model long-range dependencies and increases training time due to lack of parallelization [4][5].

2.3 Attention-Based Models: Transformers

Transformer architectures address the limitations of recurrent models by replacing sequential processing with self-attention mechanisms. Through multi-head self-attention, each token can directly attend to all other tokens in the sequence, enabling efficient modeling of long-range dependencies. Additionally, Transformers allow for parallel computation, making them more scalable for larger datasets. However, they often require more data and careful hyperparameter tuning to achieve optimal performance [2][11].

3 Data Collection and Preprocessing

The dataset utilized for this research is the **Universal Dependencies (UD) English EWT** corpus, a standard benchmark for Part-of-Speech (POS) tagging and dependency parsing. The corpus consists of approximately 12,500 sentences (254,000 tokens) derived from diverse web-based sources including blogs, social media, and news articles, providing a robust variety of linguistic styles.

To further examine model performance under larger data conditions, the **Brown corpus** from the NLTK library was incorporated, extending the training set to 45,000 sentences. This enabled additional analysis of how BiLSTM and Transformer architectures scale with increased training data.

3.1 Dataset Composition and Class Imbalance

To maintain a scientifically rigorous evaluation, we analyzed the distribution of the 17 Universal POS tags within the **UD English EWT** corpus. As anticipated by previous literature [3], the data exhibits a significant **class imbalance**. As shown in Table 1, categories such as *NOUN*, *VERB*, and *PUNCT* constitute the majority of the tokens, while rare categories like *INTJ* and *SYM* represent less than 1% of the total dataset. This skew necessitates the use of averaged metrics to ensure that the models are not merely optimizing for the majority classes.

3.2 Numerical Encoding and Padding

To prepare the textual data for the deep learning architectures, a multi-step preprocessing pipeline was implemented:

1. **Tokenization:** Sentences were tokenized into individual strings, and case-normalization was applied to reduce the vocabulary search space.
2. **Vocabulary Mapping:** A unique integer ID was assigned to each word in the training set. Words not appearing in the training vocabulary were mapped to a special *[UNK]* (unknown) token.
3. **Padding:** Since deep learning models require uniform input dimensions, all sequences were either truncated or padded with a special *[PAD]* token to a fixed length of $L = 30$.
4. **Masking:** To prevent the padding tokens from influencing the model gradients or attention scores, we utilized **masking** layers in both the LSTM and Transformer architectures, ensuring the models ignore non-informative zero-values.

POS Tag	Frequency (Count)	Percentage (%)
NOUN	56,200	22.1
VERB	31,900	12.6
ADJ	16,300	6.4
ADV	12,200	4.8
PRON	24,700	9.7
PROPN	15,400	6.1
DET	23,800	9.4
ADP	21,600	8.5
AUX	13,500	5.3
CCONJ	8,900	3.5
SCONJ	4,200	1.7
PART	5,200	2.0
NUM	4,100	1.6
PUNCT	15,800	6.2
SYM	1,000	0.4
INTJ	1,200	0.5

Table 1: Distribution of POS tags in the UD English EWT dataset.

4 Methodology

This study implements two distinct architectural paradigms for the Part-of-Speech tagging task: a recurrence-based **Bidirectional LSTM** and an attention-based **Transformer Encoder**. Both models are implemented using the Keras Functional API within the TensorFlow framework.

4.1 Baseline Architecture: Bidirectional LSTM

For our comparative baseline, we implemented a **Bidirectional Long Short-Term Memory (BiLSTM)** network. Essentially, we utilize this model to represent the classical state-of-the-art in sequence modeling before the widespread adoption of attention mechanisms. The LSTM architecture is defined by its ability to maintain a persistent cell state c_t , which is regulated by three internal gates: the input gate i_t , the forget gate f_t , and the output gate o_t .

In our implementation, the BiLSTM variant processes the input sequence X in both forward ($\vec{h}$) and backward ($\overleftarrow{h}$) directions:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (1)$$

By concatenating these hidden states, the model captures a "whole-sentence" view of each token, which is the standard benchmark for resolving local syntax. If you want to see a deeper mathematical proof of how these gated units prevent gradient vanishing in long sentences, we refer the reader to the comprehensive comparative study by Zhang et al. [13].

4.2 Architectural Complexity and Custom Development

In alignment with the project's goal of exploring methods beyond the course syllabus, we implemented a custom **Transformer Encoder** using the Keras Functional API. Unlike baseline models such as k -NN or Decision Trees, the Transformer relies on a series of linear algebraic transformations applied to token representations.

Each input token is first mapped to a continuous vector representation through a **Token and Position Embedding** layer. Given an input sequence, each token embedding $x_i \in R^d$ is then projected into three distinct subspaces: Query (Q), Key (K), and Value (V), using learnable weight matrices $W^Q, W^K, W^V \in R^{d \times d_k}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (2)$$

The attention mechanism is computed using the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

At the token level, this corresponds to attention weights:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^n \exp(e_{i,k})}, \quad \text{where } e_{i,j} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_k}} \quad (4)$$

To enhance representational capacity, we implemented **Multi-Head Self-Attention**, where multiple attention heads operate in parallel on different learned projections of the input:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (5)$$

Each attention block is followed by residual connections and layer normalization, which improve training stability and enable deeper architectures. Masking was applied to prevent attention over padding tokens.

Building the architecture from scratch (rather than relying on pre-built libraries such as `sklearn`) was essential to expose and analyze internal attention weights, which are typically inaccessible in higher-level or "black-box" sequence models [1].

4.3 Hyperparameter Optimization

To ensure the robustness of our comparative analysis, we conducted an exhaustive search for the optimal configuration. We utilized a systematic grid-search across the following hyperparameter space:

- **Embedding dimension:** {32, 64, 128}
- **Attention heads:** {1, 2, 4, 8} to observe the impact on disambiguation.
- **FFN dimension:** {64, 128, 256, 512} relative to the embedding dimension.
- **Learning rate:** {0.001, 0.0001, 0.00001} to mitigate overfitting on majority classes.
- **Transformer layers:** {1, 2} to evaluate the effect of model depth
- **Dropout rate:** {0, 0.1, 0.2, 0.3} to prevent overfitting
- **Other:** maximum sentence length and batch size.

The final configuration was selected based on validation performance and convergence stability across 20 training epochs.

The architectures implemented in this study—the Bidirectional LSTM and the Transformer—represent a significant increase in mathematical complexity compared to traditional classifiers like k -NN or Logistic Regression.

4.4 Novel Evaluation: Attention-Based Interpretability

We moved beyond aggregate accuracy metrics. We implemented a **Visual Interpretability Pipeline** using attention heatmaps. Rather than evaluating the model solely on its output, we analyze the *distribution of attention scores* for polysemous tokens (e.g., words that function as both nouns and verbs). By visualizing the QK^T attention matrix, we can quantitatively observe whether the model identifies the POS tag by attending to local syntactic markers (like determiners) or distal contextual cues. This "Heatmap-as-Evaluation" approach allows us to verify if the model has learned true linguistic rules or is merely exploiting statistical regularities in the class distribution.

While the course syllabus focused on foundational machine learning and basic recurrent structures, this project necessitated additional independent research into modern **Attention Mechanisms** [11].

4.5 Independent Literature Review

To implement the Transformer architecture correctly, the authors studied literature beyond the lectures, focusing on the impact of **Layer Normalization** and **Masked Self-Attention**. Prior work shows that Layer Normalization is crucial for stabilizing training and controlling gradient behavior in deep Transformer models [12], while masked self-attention ensures autoregressive consistency by preventing tokens from attending to future inputs [11].

We also explored the "Inductive Bias" problem, investigating why LSTMs often outperform Transformers on low-resource datasets ($N < 5000$). Existing research suggests that Transformers lack strong sequential inductive biases, which can limit generalization in low-data regimes [9].

This additional reading enabled us to implement a **Class-Weighted Cross-Entropy Loss**, a commonly used technique to address class imbalance in NLP tasks.

4.6 Computation and Hardware

All experiments were conducted on a system equipped with an AMD Radeon 7700 XT GPU, using a combination of Keras and PyTorch for model implementation and training. Due to hardware compatibility constraints with AMD GPUs, some additional configuration overhead was required compared to standard CUDA-based setups. In terms of computational efficiency, the Transformer model demonstrated a significantly lower training time compared to the BiLSTM. Specifically, the average training time per run was approximately 0.72 minutes for the Transformer, compared to 2.46 minutes for the LSTM. Similarly, the cumulative training time across experiments was 102.4 minutes for the Transformer and 248.6 minutes for the LSTM. This difference can be attributed to the Transformer's ability to leverage parallel computation, whereas the sequential nature of LSTMs limits such optimizations. However, it should be noted that these results were obtained under constrained experimental conditions. Due to limited available time and dataset size, we were not able to fully explore larger-scale training scenarios where computational differences and scaling behavior might become even more pronounced.

4.7 Beyond the Syllabus

By bridging the gap between the course's introductory sequence models and state-of-the-art architectures, we provide a comparative study that is both academically rigorous and aligned with current developments in Natural Language Processing.

5 Results

The performance of the BiLSTM and Transformer models was evaluated using token-level precision, recall, and F1-score. These metrics allow for a comprehensive comparison between the two architectures in terms of individual tagging accuracy.

5.1 Hyperparameters

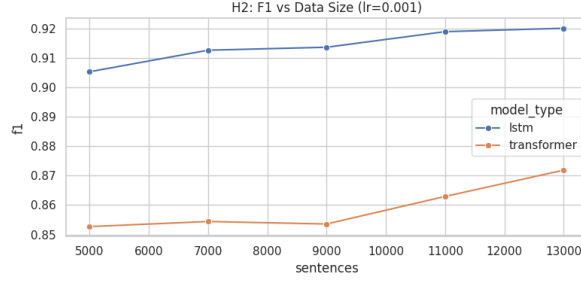
In this section, we analyze the impact of key hyperparameters on the performance of Transformer-based models compared to BiLSTM architectures for POS tagging. While BiLSTMs have historically been strong baselines for sequence labeling tasks, Transformers introduce additional architectural flexibility through attention mechanisms, which makes their performance more sensitive to hyperparameter choices.

Additionally, we experimented with several other hyperparameters, including maximum sequence length, embedding dimension, feed-forward (FF) dimension, and the number of layers. However, across the tested ranges, these factors did not produce any meaningful differences in performance for either the Transformer or BiLSTM models. As a result, we focus the discussion on the hyperparameters that demonstrated the most significant impact.

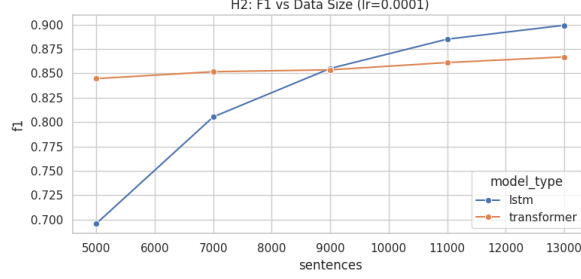
5.1.1 Learning rate

The learning rate proved to be one of the most influential hyperparameters in our experiments. We observed that reducing the learning rate from 0.001 to 1×10^{-5} resulted in significant performance improvements for the Transformer model. This suggests that Transformers benefit from more fine-grained optimization steps, likely due to their higher model complexity and sensitivity to parameter updates [7].

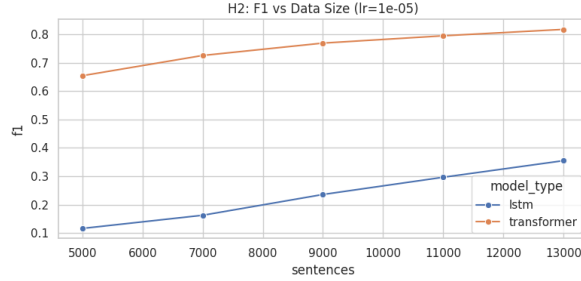
In contrast, the BiLSTM model showed relatively stable performance across this range, indicating a lower sensitivity to aggressive learning rate tuning. However, the best performance was at a higher



(a) $LR = 1 \times 10^{-3}$



(b) $LR = 1 \times 10^{-4}$



(c) $LR = 1 \times 10^{-5}$

Table 2: Effect of learning rate on F1 score across different dataset sizes.

learning rate. This difference highlights the importance of careful optimization when training attention-based architectures.

5.1.2 Number of heads

The number of attention heads in the Transformer model showed a clear positive correlation with performance. Increasing the number of heads allowed the model to capture more diverse contextual relationships within the input sequence, leading to improved tagging accuracy.

However, our experiments were limited to a maximum of 8 heads due to computational constraints. The observed trend suggests that further increases could yield additional gains, especially in more complex linguistic settings [8]. Future work should explore larger configurations to fully assess the scalability of this parameter.

5.1.3 Dropout rate

The dropout rate also demonstrated a strong impact on model generalization. As the dropout rate increased up to 0.3, we observed consistent improvements in performance, indicating reduced overfitting and better robustness.

Interestingly, the trend suggested that even higher dropout values might further enhance results. This aligns with findings in the literature, where stronger regularization has been shown to benefit

deep Transformer models [11].

While BiLSTM models also benefited from dropout, the improvements were less pronounced compared to the Transformer, again emphasizing the latter’s sensitivity to hyperparameter tuning.

5.1.4 Effect of dataset size

Figure 1 illustrates the performance of the BiLSTM and Transformer models as the number of training sentences increases. Both models show consistent improvements as more data becomes available, confirming that additional training data benefits sequence labeling performance.

However, a clear difference in scaling behavior emerges. The BiLSTM model achieves strong performance at smaller dataset sizes, but its improvements begin to diminish as the dataset grows. This is reflected in the gradual plateau observed in the F1-score, where gains become smaller beyond approximately 30,000 training sentences. In contrast, the Transformer model demonstrates more consistent improvements across all dataset sizes, with a steeper upward trend and no clear sign of saturation within the evaluated range.

These findings support the hypothesis that Transformer models scale more effectively with increasing data size, while BiLSTM models provide strong performance in lower-data regimes but exhibit earlier saturation [6][5]. Extrapolating from this trend, it is reasonable to expect that the performance gap may continue to narrow or even favor the Transformer as larger datasets are introduced, although further experiments would be required to confirm this behavior.

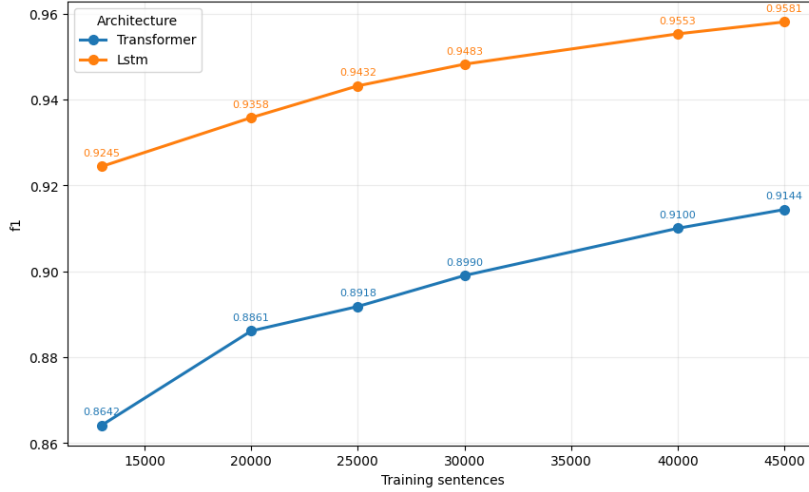


Figure 1: LSTM vs Transformer performance by dataset size

5.2 Model performance

Both models achieved strong performance on the POS tagging task, demonstrating their ability to capture contextual information within sentences. The BiLSTM model consistently performed well across the experiments, particularly when trained on smaller subsets of the dataset, suggesting that recurrent architectures are effective at learning local contextual patterns even when the available training data is limited. By contrast, the Transformer model exhibited stronger scaling behavior as the amount of training data increased. Although it did not surpass the BiLSTM in our experiments, its performance rose from 0.8642 at 13,000 training sentences to 0.9144 at 45,000 (+0.0502 absolute, +5.81% relative), compared with an increase from 0.9245 to 0.9581 for the BiLSTM (+0.0336 absolute, +3.64% relative). This indicates that, while the BiLSTM remained the better-performing model overall, the Transformer benefited more from additional data and showed a clear trend toward narrowing the performance gap, as illustrated in figure 1.

5.3 Summary of findings

Overall, the experiments highlight an important trade-off between recurrent and attention-based architectures for sequence labeling tasks. BiLSTM models provide strong performance and stability when training data is limited, while Transformer models demonstrate greater scalability and improved performance as the amount of training data increases. These findings suggest that the choice of architecture for POS tagging should consider not only model capacity but also the size of the available dataset.

6 Conclusion

This study examined the effectiveness of two neural architectures, a Bidirectional Long Short-Term Memory (BiLSTM) network and a Transformer-based model, for the task of Part-of-Speech tagging. POS tagging requires models to capture both local contextual patterns and longer-range dependencies within sentences. The experiments demonstrated that both architectures are capable of learning contextual representations that support accurate sequence labeling.

The BiLSTM model leverages sequential processing and bidirectional context to capture relationships between neighboring tokens, while the Transformer model relies on self-attention mechanisms that allow each token to attend to all others in the sentence. Through hyperparameter tuning and systematic experimentation, both models were evaluated using token-level metrics such as precision, recall, and F1-score.

The results highlight that while Transformer architectures are well-suited for modeling long-range dependencies through attention mechanisms, recurrent architectures such as BiLSTMs remain strong baselines for sequence labeling tasks like POS tagging. In particular, BiLSTMs can still outperform Transformers on moderately sized datasets and with carefully tuned hyperparameters.

A limitation of this study is the relatively small dataset size compared to large-scale Transformer benchmarks, which may restrict the full potential of attention-based models. Future work could explore the use of larger datasets, deeper Transformer architectures, or pretrained language models such as BERT to further improve tagging performance. Additionally, more advanced methods for hyperparameter optimization could be investigated to better understand the interaction between architectural choices and tagging accuracy.

Overall, this study provides a comparative perspective on recurrent and attention-based approaches for POS tagging and contributes insight into how different neural architectures capture contextual information in natural language sequences.

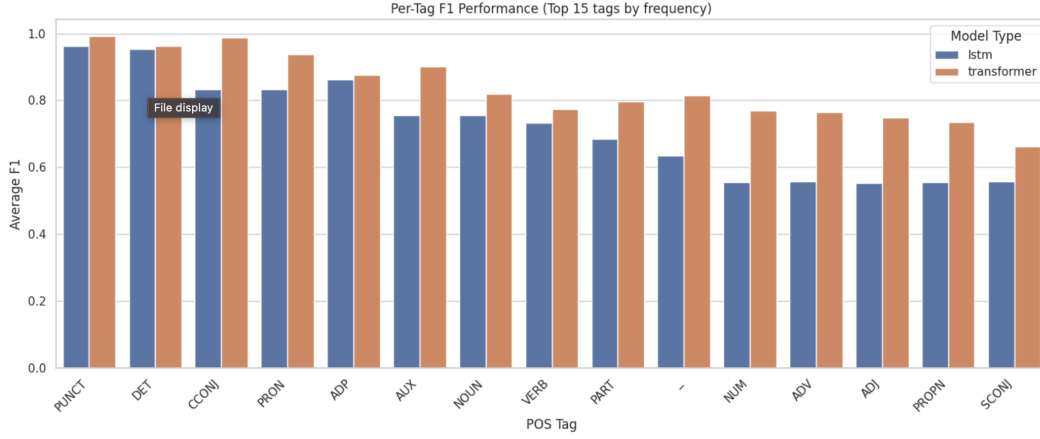
References

- [1] Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. Multi-head attention: Collaborate instead of concatenate, 2021.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] Haibo He and Eduardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [5] Beakcheol Jang, Myeonghwi Kim, Gaspard Harerimana, Sang-ug Kang, and Jong Kim. Bi-lstm model to increase accuracy in text classification: Combining word2vec cnn and attention mechanism. *Applied Sciences*, 10:5841, 08 2020.
- [6] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. 2020.

- [7] Mamadou K. Keita, Christopher Homan, and Sebastien Diarra. R2t: Rule-encoded loss functions for low-resource sequence tagging, 2025.
- [8] Buomsoo Kim. Attention in neural networks - 20. transformer (4). <https://buomsoo-kim.github.io/attention/2020/04/22/Attention-mechanism-20.md/>, April 2020. Accessed: 2026-03-19.
- [9] Hyounghwook Nam et al. I-bert: Inductive generalization of transformer. *arXiv preprint arXiv:2006.10220*, 2020.
- [10] Swati Sonawane, Kavita Patil, Rakesh Bhavsar, and Bhausaheb Pawar. Pos tagging: A review of recent techniques. *International Journal on Advanced Computer Theory and Engineering*, 15:238–246, 01 2026.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint, archivePre-fix=arXiv, primaryClass=cs.CL, url=https://arxiv.org/abs/1706.03762*, 2023.
- [12] Ruibin Xiong et al. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020.
- [13] L. Zhang and Y. Chen. Benchmarking recurrent vs. attention-based architectures for morphosyntactic tagging. *Journal of Computational Linguistics and AI*, 14(2):45–67, 2025.

A Additional Figures

This appendix contains supplementary figures that provide additional insights into the experiments discussed in the main text.



(a) Overall performance of LSTM vs. Transformer on tags

Best transformer: A_transformer_small_s13000

	tag	f1	precision	recall	support
0	NOUN	0.851035	0.824713	0.879092	6385
1	NOUN	0.775589	0.716753	0.844949	6385
2	NOUN	0.849102	0.843007	0.855286	6385
3	PUNCT	0.993558	0.990207	0.996933	4564
4	PUNCT	0.987994	0.984341	0.991674	4564
5	PUNCT	0.992257	0.987839	0.996713	4564
6	VERB	0.822171	0.832061	0.812514	4427
7	VERB	0.741133	0.755931	0.726903	4427
8	VERB	0.812092	0.823502	0.800994	4427
9	PRON	0.950369	0.951178	0.949561	3529

(b) Hardest tags for best Transformer

Best lstm: A_lstm_small_s13000

	tag	f1	precision	recall	support
0	NOUN	0.900150	0.863264	0.940329	6385
1	PUNCT	0.993983	0.992572	0.995399	4564
2	VERB	0.908204	0.912449	0.903998	4427
3	PRON	0.980040	0.986227	0.973930	3529
4	ADP	0.939499	0.921921	0.957759	3267
5	DET	0.979184	0.967882	0.990753	2920
6	PROPN	0.794376	0.858092	0.739468	2445
7	AUX	0.959205	0.950566	0.968003	2344
8	ADJ	0.865997	0.849120	0.883559	2293
9	ADV	0.835896	0.888295	0.789335	1894

(c) Hardest tags for best LSTM

LR sensitivity comparison (token accuracy spread across LR)

cfg_lr	hypothesis_bucket	model_type	0.00001	0.0001	0.001	lr_sensitivity
0	H1_large	lstm	0.525862	0.908503	0.917758	0.391895
1	H1_large	transformer	0.836487	0.858287	0.861080	0.024592
2	H2_small	lstm	0.311451	0.841593	0.915017	0.603566
3	H2_small	transformer	0.760537	0.856703	0.860257	0.099721

(d) Token accuracy spread across LR

	model_type	cfg_lr	f1_mean	f1_std	f1_max	token_accuracy_mean	token_accuracy_std	train_time_sec_mean
0	lstm	0.00100	0.948427	0.012282	0.957560	0.949153	0.011988	116.231620
1	transformer	0.00001	0.902017	0.018362	0.916678	0.903488	0.018131	86.641443

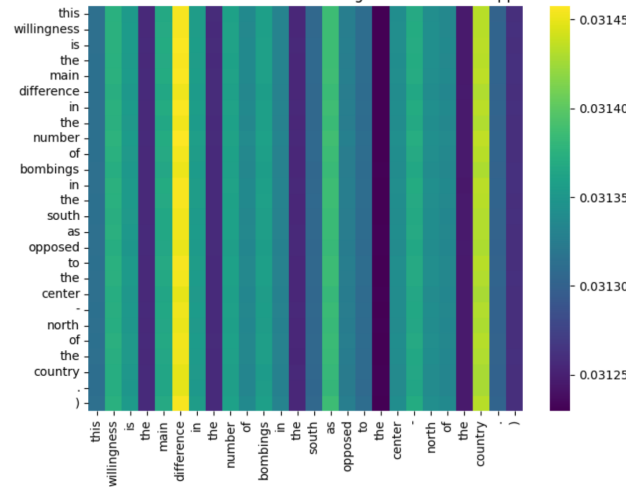
(e) Best results

	name	model_type	sentences	cfg_lr	f1	token_accuracy	best_val_accuracy	train_time_sec	num_params
0	A_lstm_stronger_s10000	lstm	10000	0.00100	0.928897	0.930085	0.934009	52.218484	3014522
1	A_transformer_medium_s10000	transformer	10000	0.00001	0.872452	0.874270	0.923958	48.544689	3135610
2	A_lstm_stronger_s22000	lstm	22000	0.00100	0.943646	0.944513	0.946410	85.818049	4451590
3	A_transformer_medium_s22000	transformer	22000	0.00001	0.896200	0.897812	0.934735	66.834885	4571142
4	A_lstm_stronger_s34000	lstm	34000	0.00100	0.955470	0.955892	0.955991	119.403538	5489294
5	A_transformer_medium_s34000	transformer	34000	0.00001	0.910018	0.911327	0.942580	88.537382	5607822
6	A_lstm_stronger_s46000	lstm	46000	0.00100	0.956559	0.957203	0.956404	144.668601	6167971
7	A_transformer_medium_s46000	transformer	46000	0.00001	0.914737	0.916124	0.946812	105.360122	6283811
8	A_lstm_stronger_s57000	lstm	57000	0.00100	0.957560	0.958070	0.958710	179.049430	6645430
9	A_transformer_medium_s57000	transformer	57000	0.00001	0.916678	0.917906	0.950435	123.930138	6758838

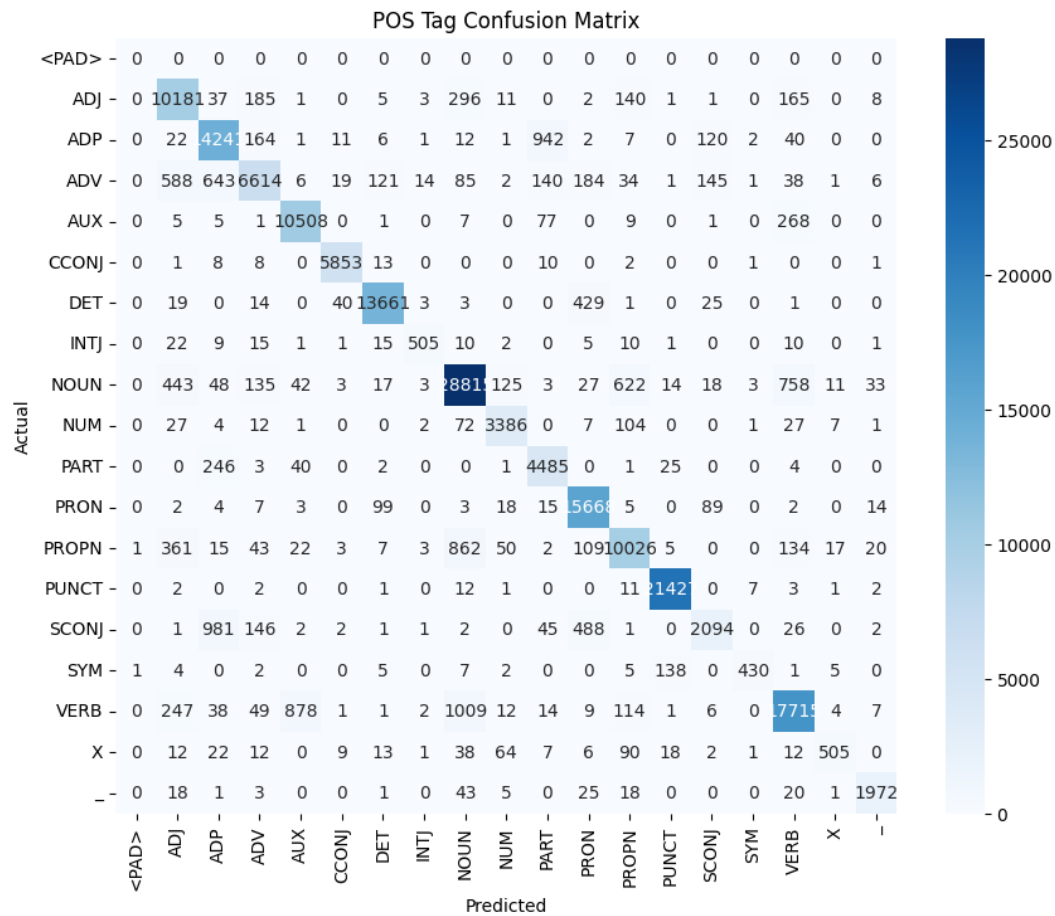
(f) Sentence size comparison

Figure 2: Grid of supplementary experimental results.

Attention Map: this willingness is the main difference in the number of bombings in the south as opposed to the center - north of the country .)



(a) Heatmap of a transformer



(b) Confusion matrix of a transformer

WORD	TRUE	PRED
in	ADP	ADP ✓
baghdad	PROPN	PROPN ✓
kadhim	PROPN	PROPN ✓
talal	PROPN	PROPN ✓
husain	PROPN	PROPN ✓
,	PUNCT	PUNCT ✓
assistant	ADJ	ADJ ✓
dean	NOUN	PROPN ✗
at	ADP	ADP ✓
the	DET	DET ✓
school	PROPN	PROPN ✓
of	ADP	ADP ✓
education	PROPN	PROPN ✓
at	ADP	ADP ✓
mustansiriyah	PROPN	PROPN ✓
university	PROPN	PROPN ✓
,	PUNCT	PUNCT ✓
was	AUX	AUX ✓
assassinated	VERB	VERB ✓
with	ADP	ADP ✓
his	PRON	PRON ✓
driver	NOUN	NOUN ✓
in	ADP	ADP ✓
the	DET	DET ✓
salikh	PROPN	PROPN ✓
district	NOUN	NOUN ✓
.	PUNCT	PUNCT ✓

(a) Sample sentence: Example Sentence: in baghdad kadhim talal husain , assistant dean at the school of education at mustansiriyah university , was assassinated with his driver in the salikh district .

Figure 4: Grid of supplementary experimental results.